



Saint Columban College
College of Computing Studies
Pagadian City



Glitch Cat

Glitch Cat 



Easy General Skills Beginner picoMini 2022 nc shell Python

AUTHOR: LT 'SYREAL' JONES

Description

Our flag printing service has started glitching!

```
$ nc saturn.picoctf.net 55593
```

This challenge launches an instance on demand.

Its current status is: **RUNNING**

Instance Time Remaining: **14:46**

[Restart Instance](#)

Hints

1 2 3

ASCII is one of the most common encodings used in programming

59,422 users solved



78%

Liked



 picoCTF{FLAG}

[Submit](#)

Author: The Analyst: Hyposelenia

Challenge: Glitch Cat

Category: General Skills

Date: 01/16/26



I. Objective

The objective of this challenge is to reconstruct the correct picoCTF flag from a “glitched” output by understanding ASCII encoding and recognizing valid Python syntax.

II. Background

In programming, characters are often represented internally using **ASCII values**. Python allows characters to be generated using the `chr()` function, which converts a number into its corresponding ASCII character.

This challenge presents a partially readable flag combined with ASCII character conversions. By understanding how ASCII encoding works—or by recognizing that the output is valid Python code—the original flag can be reconstructed.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment
- **Linux Terminal** – Used to execute decoding commands
- **python3** – Used to evaluate Python expressions

IV. Methodology

Method 1: Manual ASCII Conversion

1. Launched the challenge instance and connected to the remote machine.
2. The following output was displayed:

```
'picoCTF{gl17ch_m3_n07_' + chr(0x62) + chr(0x64) + chr(0x61) +  
chr(0x36) + chr(0x38) + chr(0x66) + chr(0x37) + chr(0x35) + '}'
```
3. Identified the `chr(0x..)` values as **hexadecimal ASCII codes**.



4. Converted each hexadecimal value to its ASCII equivalent using an online hex-to-ASCII converter.

rapidtables.com/convert/number/ascii-hex-bin-dec-converter.html

Home > Conversion > Number conversion > ASCII,hex,binary,decimal,base64 converter

ASCII, Hex, Binary, Decimal, Base64 converter

Enter [ASCII text](#) or hex/binary/decimal numbers:

Open File

Reset

Number delimiter

Space

☐ 0x/0b prefix

ASCII text

bda68f75

Hex (bytes)

chr(0x62) + chr(0x64) + chr(0x61) + chr(0x36) + chr(0x38) +
chr(0x66) + chr(0x37) + chr(0x35)

Binary (bytes)

00001100 01100010 00001100 01100100 00001100 01100001 00001100
00110110 00001100 00111000 00001100 01100110 00001100 00110111

Decimal (bytes)

5. Appended the decoded characters to reconstruct the full flag.



Method 2: Executing the Python Expression (More Efficient)

1. Connected to the remote machine and observed the same output.
2. The hint stated that the glitch output is **valid Python**, meaning it can be executed directly.
3. Started the Python interpreter by typing: `python3`
4. Pasted the entire glitched output into the Python prompt.
5. Python evaluated the expression and printed the complete flag automatically.

V. Result

picoCTF{gl17ch_m3_n07_bda68f75}

```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ nc saturn.picoctf.net 64866  
'picoCTF{gl17ch_m3_n07_' + chr(0x62) + chr(0x64) + chr(0x61) + chr(0x36) + chr(0x38) + chr(0x66) + chr(0x37) + chr(0x35) + '}'  
(kali@kali)-[~]  
$ python3  
Python 3.13.3 (main, Apr 10 2025, 21:38:51) [GCC 14.2.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> 'picoCTF{gl17ch_m3_n07_' + chr(0x62) + chr(0x64) + chr(0x61) + chr(0x36) + chr(0x38) + chr(0x66) + chr(0x37) + chr(0x35) + '}'  
...  
'picoCTF{gl17ch_m3_n07_bda68f75}'  
>>> █
```

VI. Explanation

This challenge works because Python allows strings to be concatenated using the `+` operator and characters to be generated using the `chr()` function. Each `chr(0x..)` call converts a hexadecimal value into its ASCII character.

- **Method 1** works by manually decoding each character, which helps beginners understand ASCII encoding.
- **Method 2** is more efficient because Python already knows how to interpret the expression. Running it directly lets Python do the decoding for you.

The hint about ASCII explains *what* the values represent, while the hint about valid Python explains *how* to solve the challenge faster.



VII. Conclusion

Glitch Cat teaches two important beginner concepts: ASCII encoding and the power of recognizing valid code syntax. While manual decoding helps build foundational understanding, executing valid Python expressions is faster and more practical in real scenarios. This challenge encourages thinking both logically and creatively when analyzing unusual outputs.

— **The Analyst: Hyposelenia**